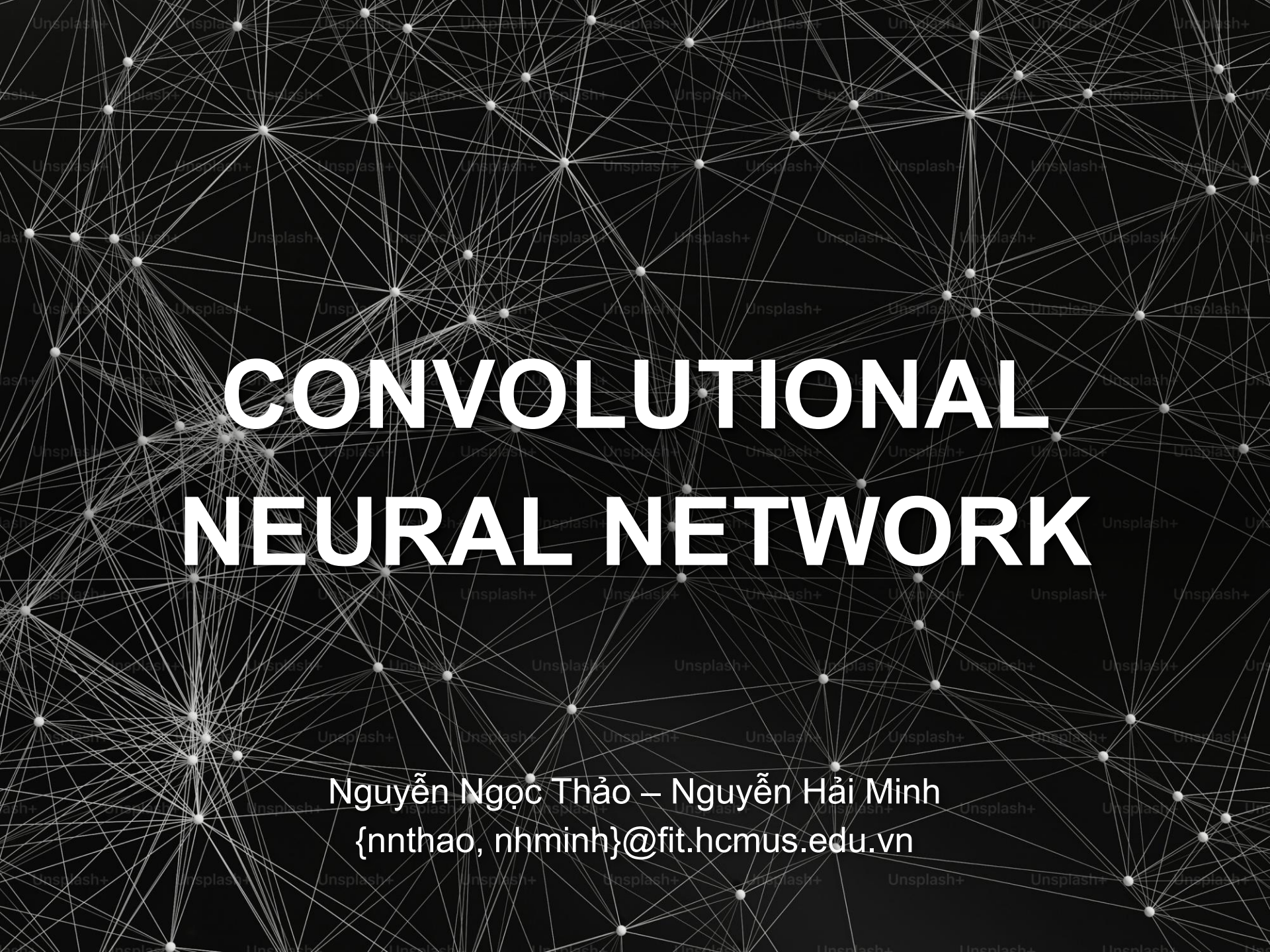


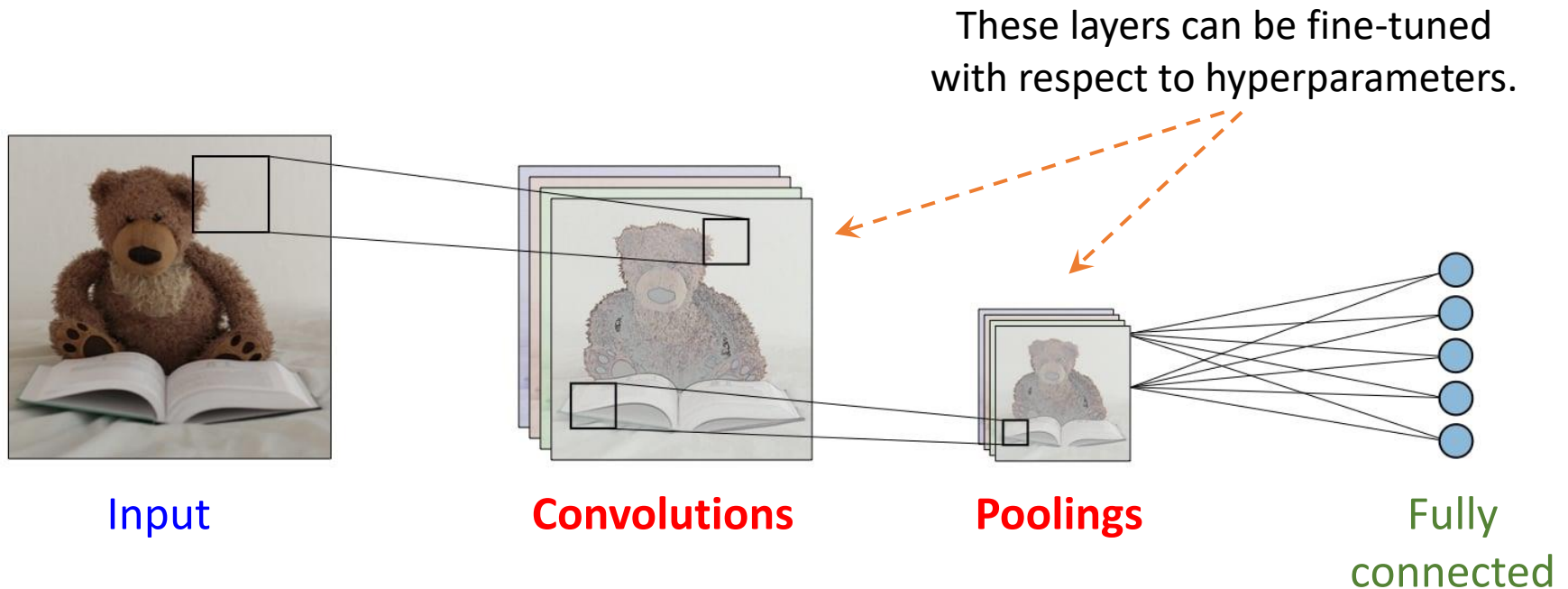


CONVOLUTIONAL NEURAL NETWORK

Nguyễn Ngọc Thảo – Nguyễn Hải Minh
{nnthao, nhminh}@fit.hcmus.edu.vn

Convolutional neural network

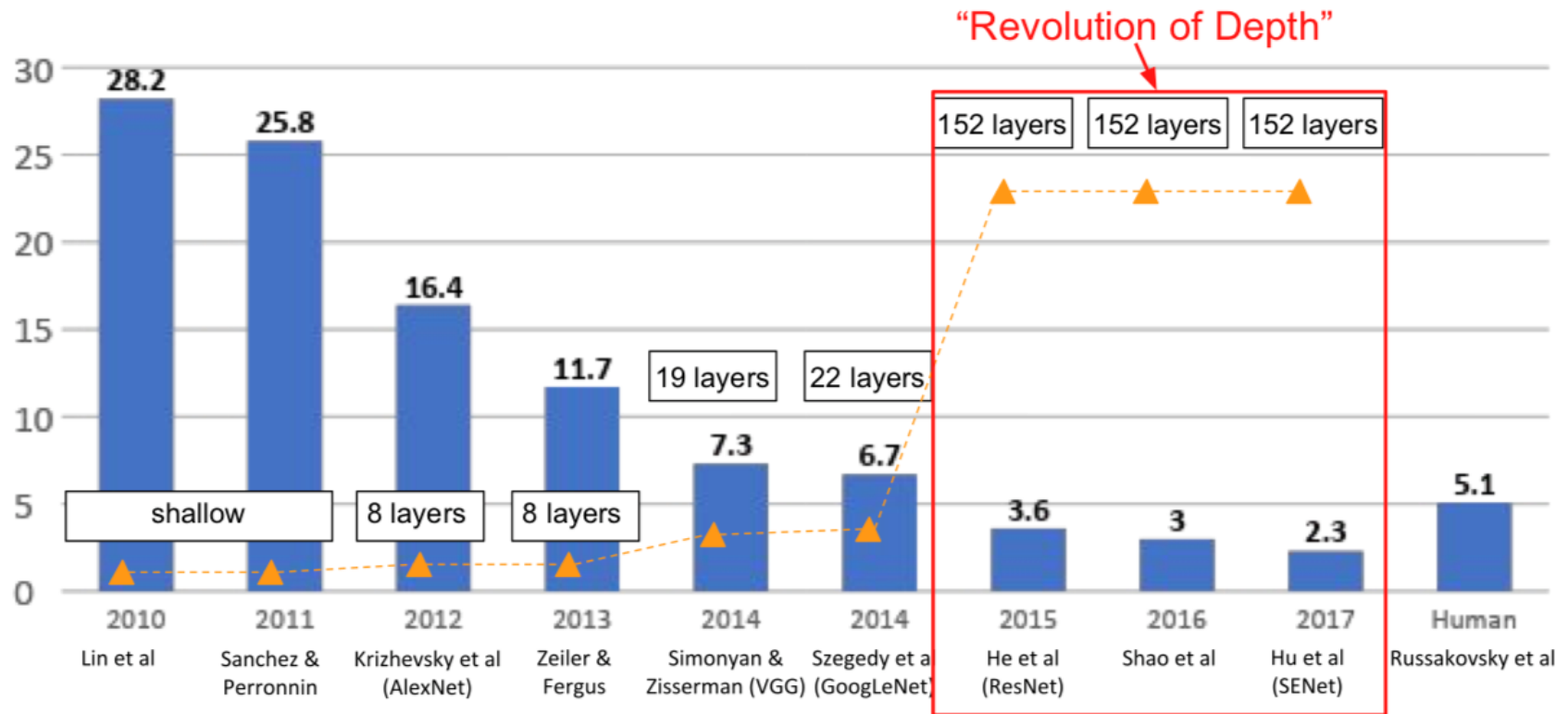
- **Convolutional neural networks** are a specific type of neural networks typically built from the following layers:



- Common abbreviations: **ConvNet** or **CNN**.

Convolutional neural network

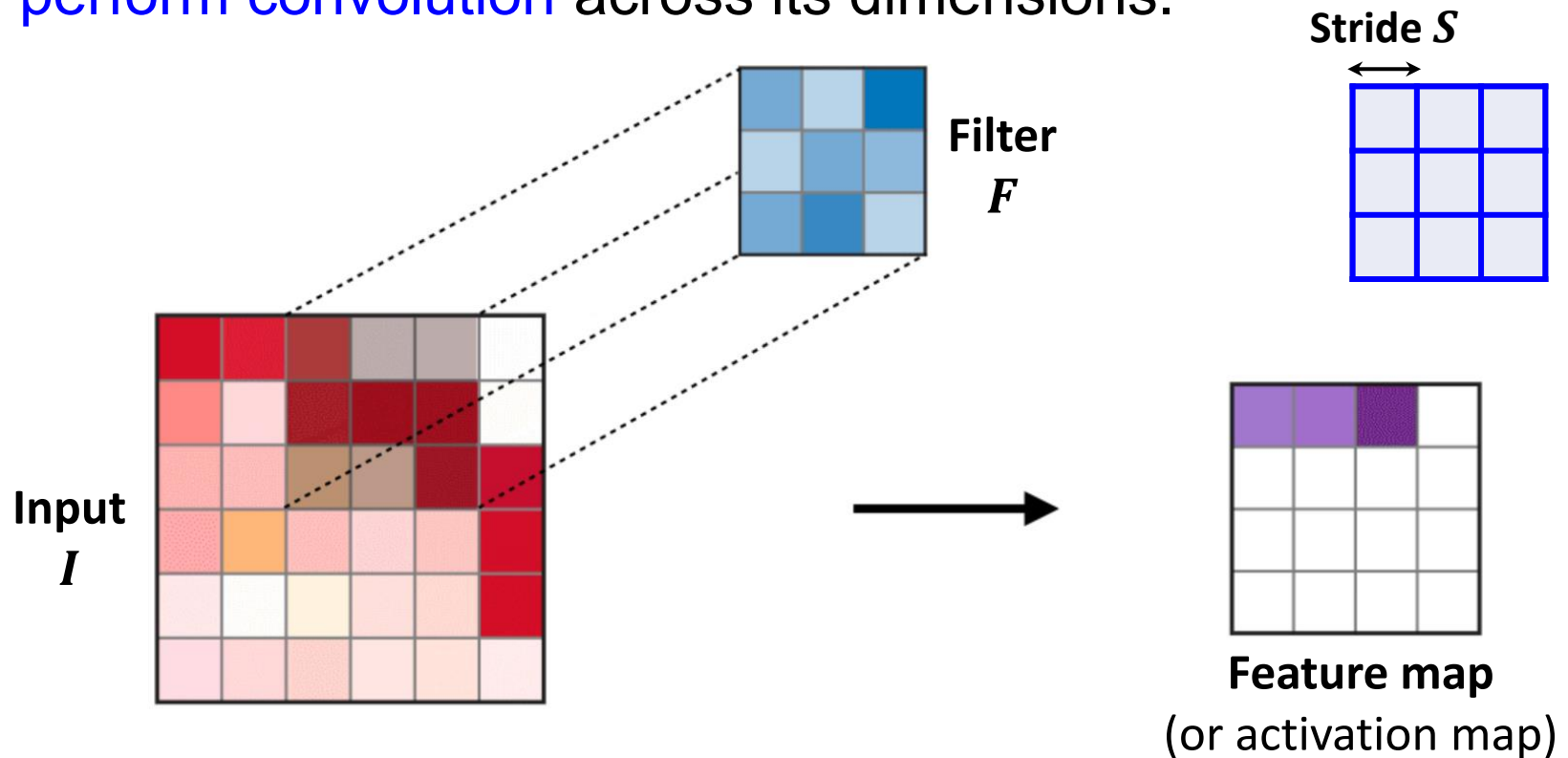
- One of the most influential innovations in computer vision



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners
(in % error rate)

Architecture: Convolutional layer

- The **convolution layer** applies **filters** that **scan the input I** and **perform convolution** across its dimensions.

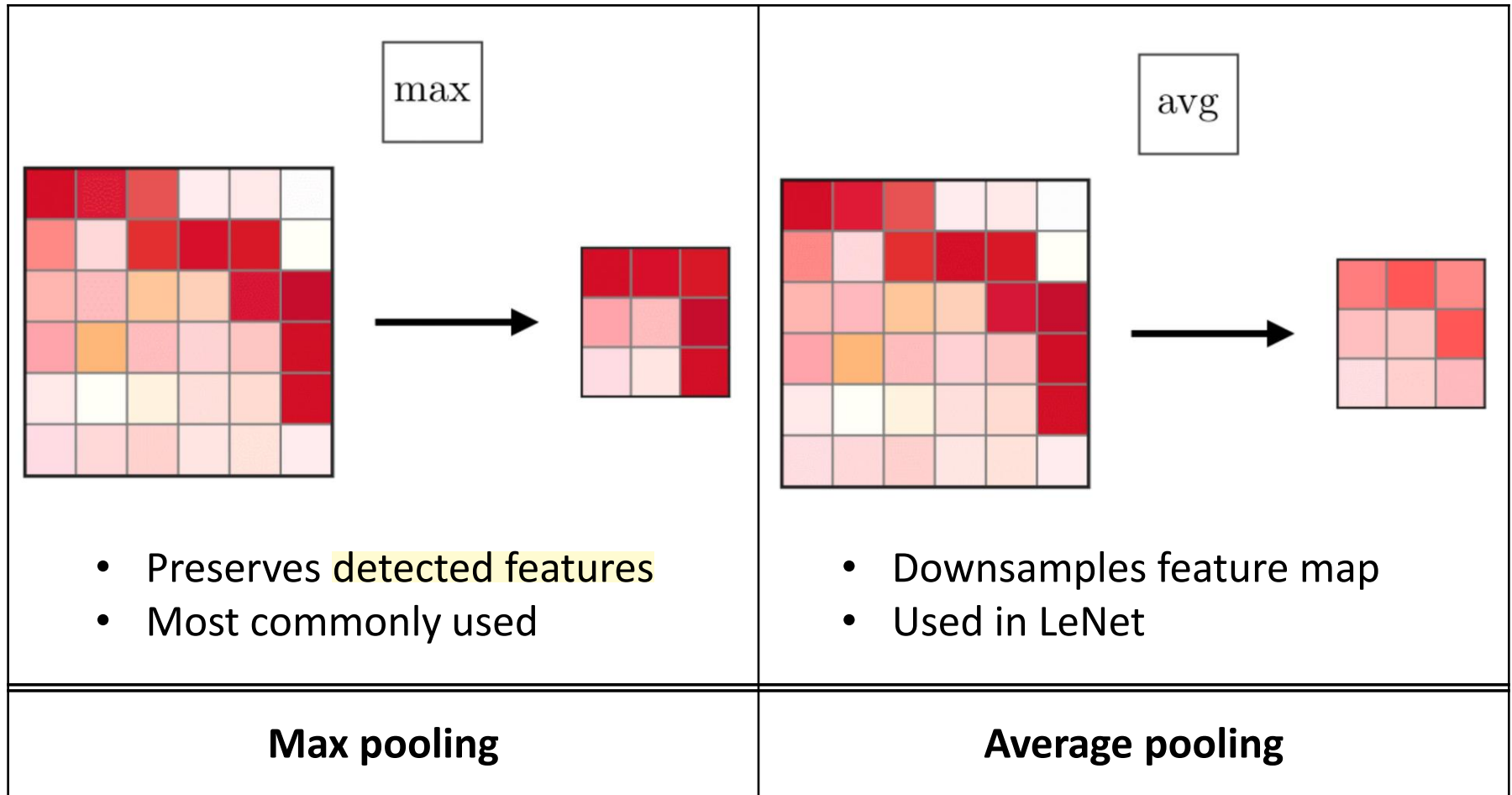


- Remark: the convolution step can be generalized to the 1D and 3D cases.*

Architecture: Pooling layer

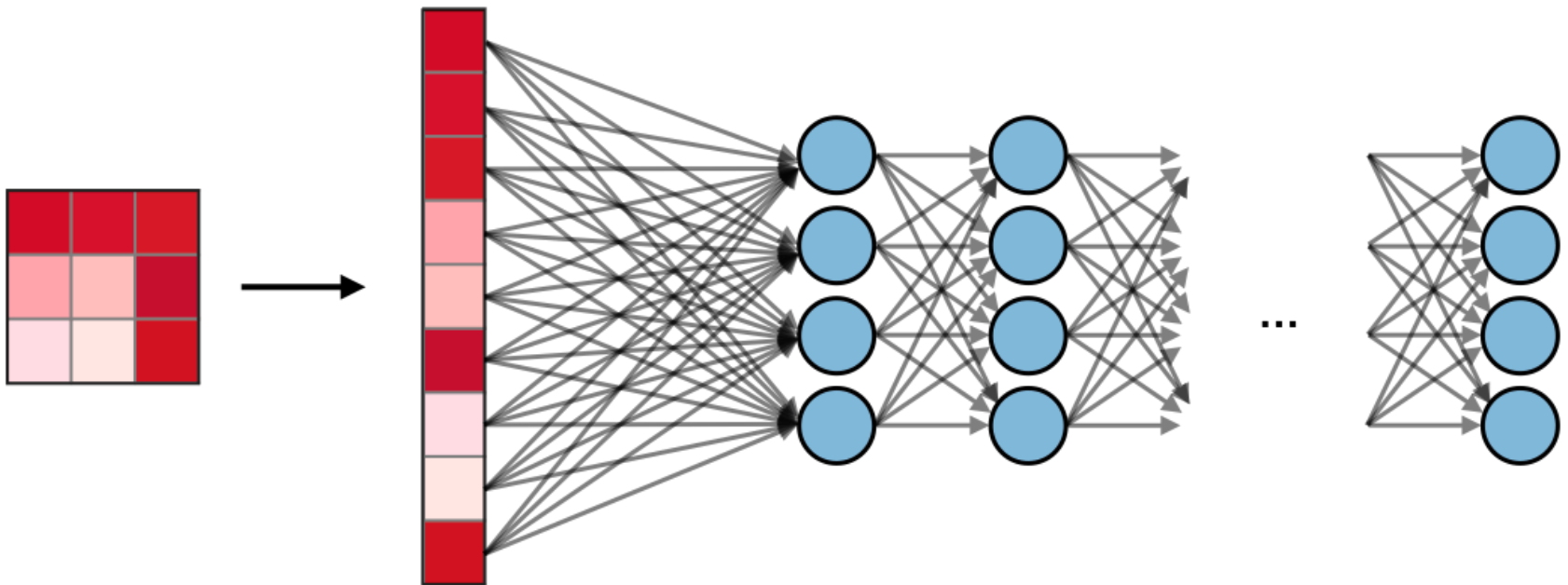
- The **pooling layer** **downsamples the feature maps** after convolution, giving the model some spatial invariance.
- It operates between two successive convolutional layers, independently on every depth slice of the input.
- **Reduce the amount of parameters, control overfitting**
- Types: Max pooling, average pooling, $\mathcal{L}_2$ -norm pooling

Architecture: Pooling layer



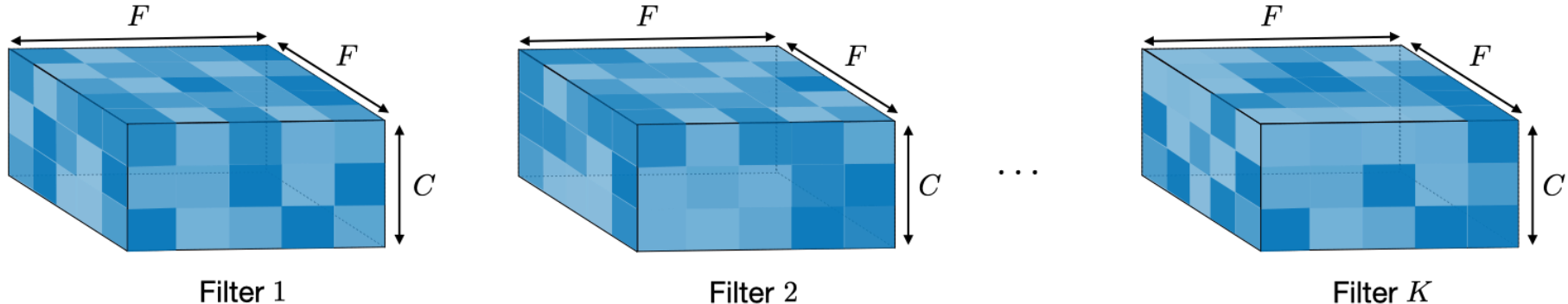
Architecture: Fully connected layer

- The **fully connected layer** (FC) **operates on a flattened input** where each input is connected to all neurons.
- They usually appear near the **end of CNNs** and are used to **optimize objectives** such as class scores.



Filter hyperparameters: Dimensions

- A filter of size $F \times F$ applied to an input with C channels is a $F \times F \times C$ volume that
 - performs convolutions on an input of size $I \times I \times C$, and
 - produces an output feature map of size $O \times O \times 1$.

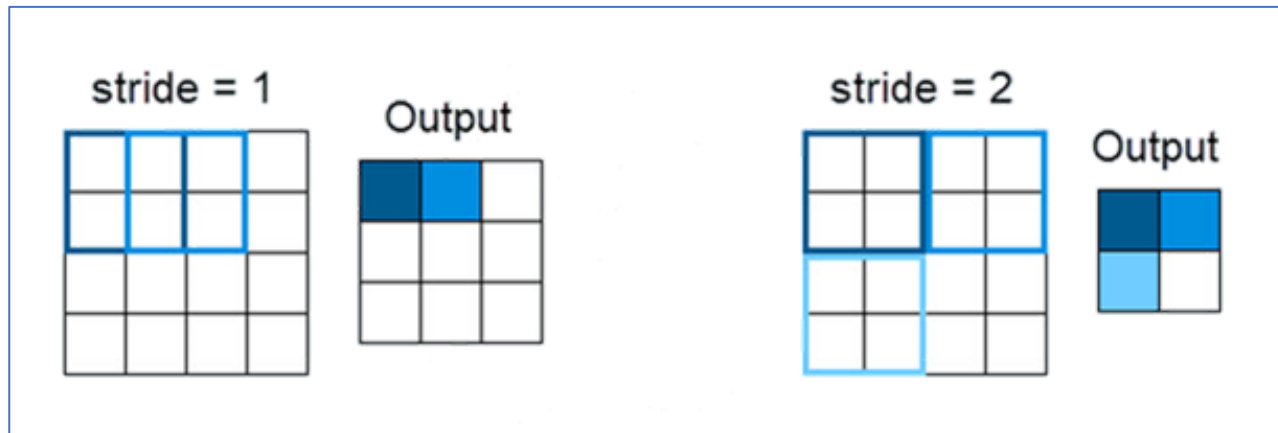
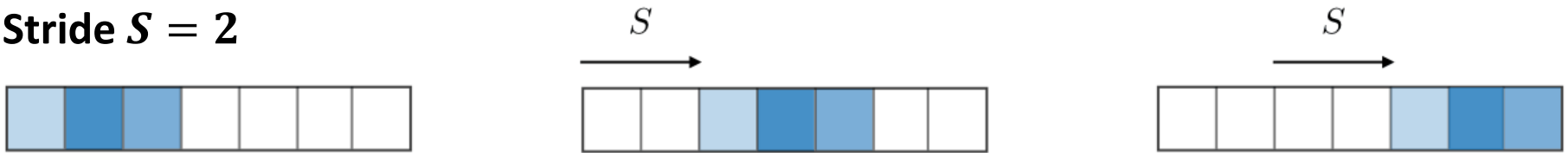


- Remark: the application of K filters of size $F \times F$ results in an output feature map of size $O \times O \times K$.

Filter hyperparameters: Stride

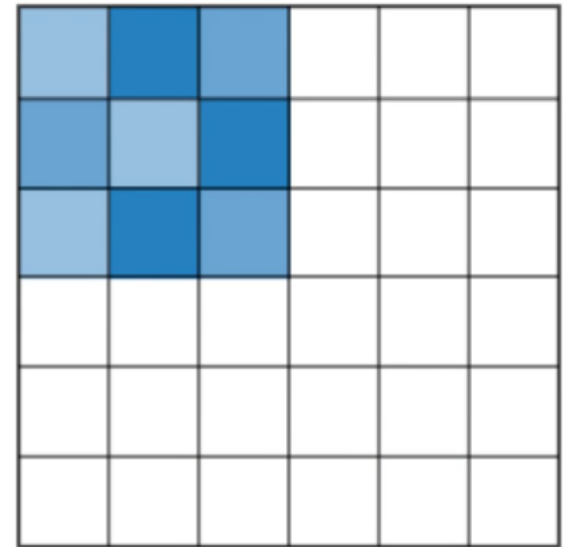
- The **stride S** denotes the **number of pixels by which the window moves** after each operation.

Stride $S = 2$



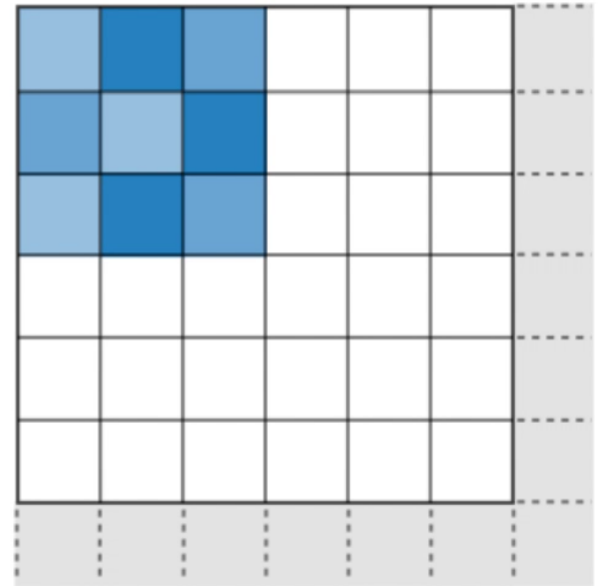
Filter hyperparameters: Zero-padding

- **Zero-padding** denotes the process of **adding P zeroes to each side of the boundaries** of the input.
- This value can be manually specified or automatically set.
- **Valid mode: $P = 0$**
 - No padding
 - **Drops last convolution if dimensions do not match**



Filter hyperparameters: Zero-padding

- **Same mode:** $P = \left\lceil \frac{s \left\lceil \frac{I}{s} \right\rceil - I + F - s}{2} \right\rceil$
 - Padding such that feature map size has size $\left\lceil \frac{I}{s} \right\rceil$
 - Output size is mathematically convenient

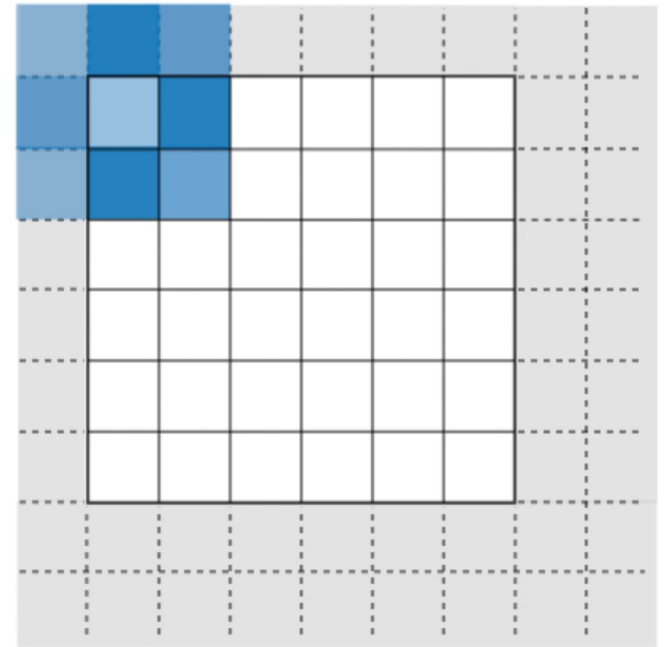


Filter hyperparameters: Zero-padding

- **Full mode:** $P_{start} \in \llbracket 0, F - 1 \rrbracket$

$$P_{end} = F - 1$$

- Maximum padding such that end convolutions are applied on the limits of the input
- Filter 'sees' the input end-to-end



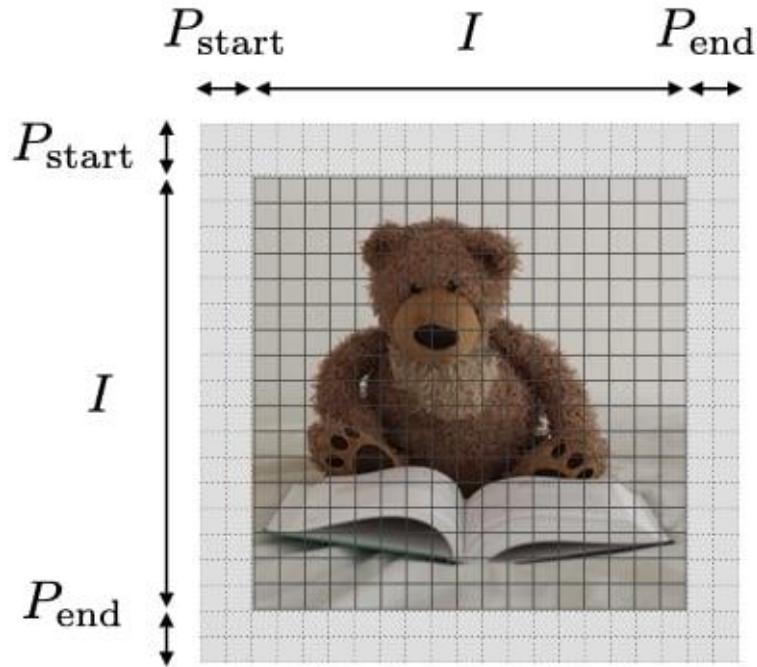
Tuning hyperparameters

- Let I the length of the input volume size, F the length of the filter, P the amount of zero padding, and S the stride.
- The output size O of the feature map along that dimension is

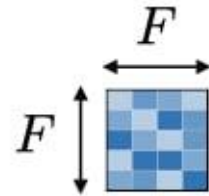
$$O = \frac{I - F + P_{start} + P_{end}}{S} + 1$$

- *Remark: often times, $P_{start} = P_{end} \triangleq P$, thus we can replace $P_{start} + P_{end}$ by $2P$ in the formula above.*

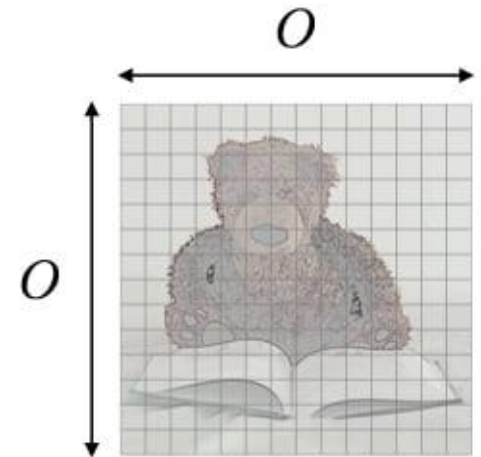
Tuning hyperparameters



Input

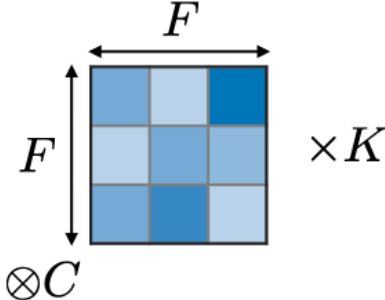
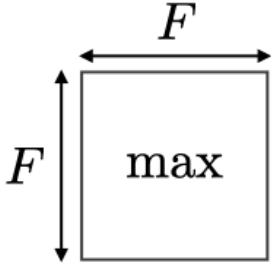
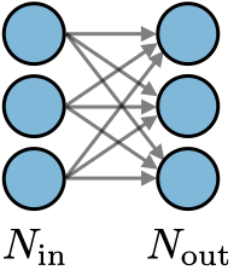


Filter



Output

The complexity of the model

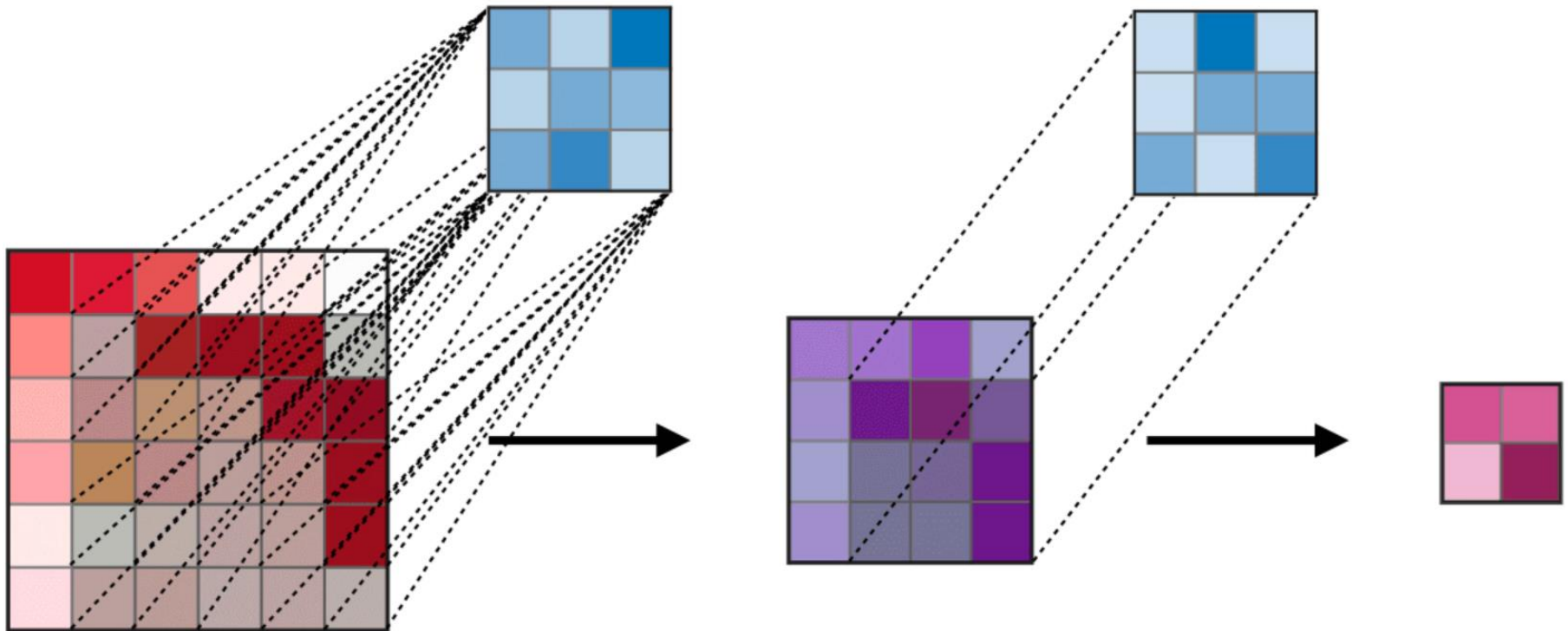
	Convolution	Pooling	FC
Illustration	 A 3x3 grid of blue squares representing a convolution kernel of size $F \times F$. The grid is labeled $\otimes C$ on the left and $\times K$ on the right.	 A square box of size $F \times F$ with the word "max" inside, representing a max pooling operation.	 A diagram showing a fully connected layer with N_{in} input nodes on the left and N_{out} output nodes on the right, connected by arrows.
Input size	$I \times I \times C$	$I \times I \times C$	N_{in}
Output size	$O \times O \times K$	$O \times O \times C$	N_{out}
Number of parameters	$(I \times I \times C) \cdot K$	0	$(N_{in} + 1) \times N_{out}$
Remarks	- One bias param per filter - In most cases, $S < F$ - A common choice for K is $2C$.	- Pooling operation done channel-wise - In most cases, $S = F$	- Input is flattened. - One bias param per neuron. - The number of FC neurons is free of structural constraints.

Receptive field

- The receptive field at layer k is the area denoted $R_k \times R_k$ of the input that each pixel of the k -th activation map can “see”.
- Let F_j be the filter size of layer j , S_i the stride of layer i and with the convention $S_0 = 1$.
- The receptive field at layer k can be computed as:

$$R_k = 1 + \sum_{j=1}^k (F_j - 1) \prod_{i=0}^{j-1} S_i$$

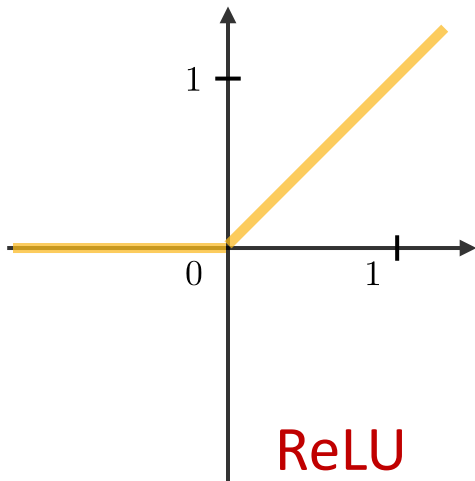
Receptive field



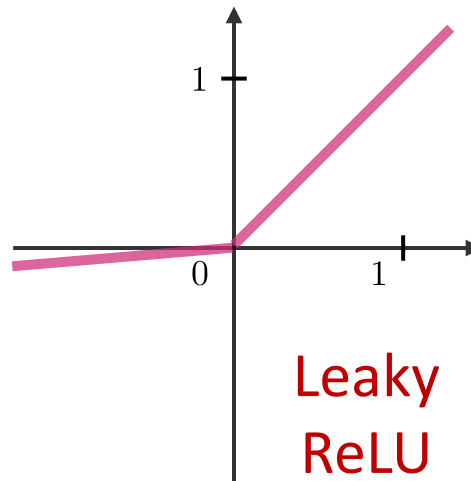
$$F_1 = F_2 = 3 \text{ and } S_1 = S_2 = 1, \text{ which gives } R_2 = 1 + 2 \cdot 1 + 2 \cdot 1 = 5$$

Common activation functions

- **Rectified linear unit layer** (ReLU) is applied on all elements of the volume, **introducing non-linearities** to the network.

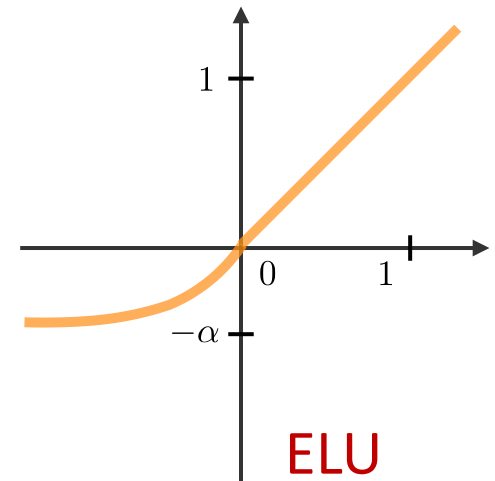


$$g(z) = \max(0, z)$$



$$g(z) = \max(\epsilon z, z)$$

with $\epsilon \ll 1$



$$g(z) = \max(\alpha(e^z - 1), z)$$

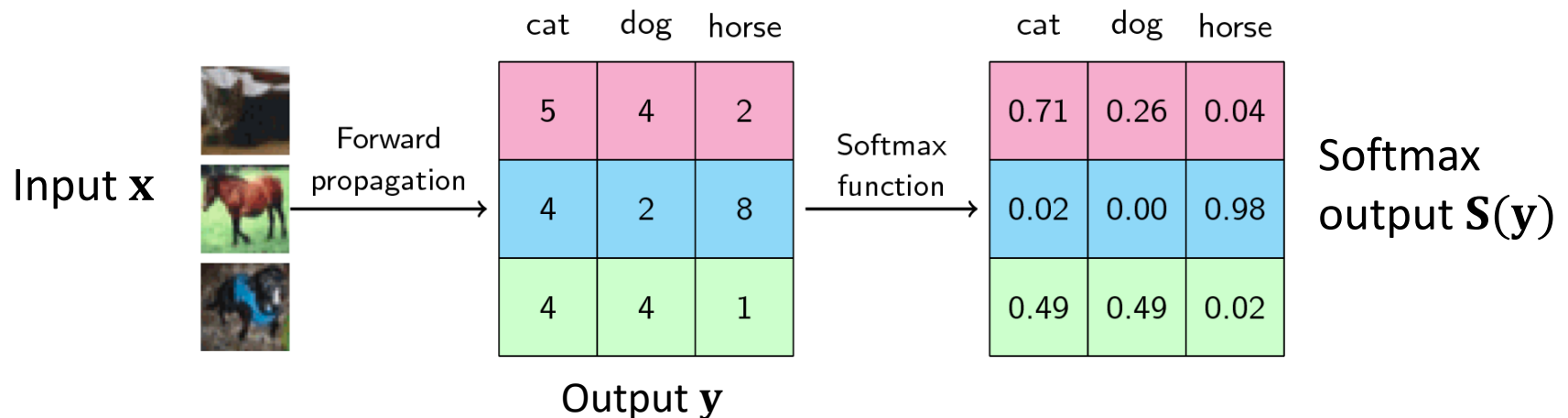
with $\alpha \ll 1$

- Non-linearity complexities
- Biologically interpretable
- Addresses dying ReLU issue for negative values
- **Differentiable everywhere**

Softmax function

- **Softmax** is applied at the end of the architecture.
- Input: a vector of scores $x \in \mathbb{R}^n$
- Output: a vector of output probability $p \in x \in \mathbb{R}^n$

- Formula: $\mathbf{p} = \begin{pmatrix} p_1 \\ \vdots \\ p_n \end{pmatrix}$ where $p_i = \frac{e^{x_i}}{\sum_{j=1}^n x_j}$

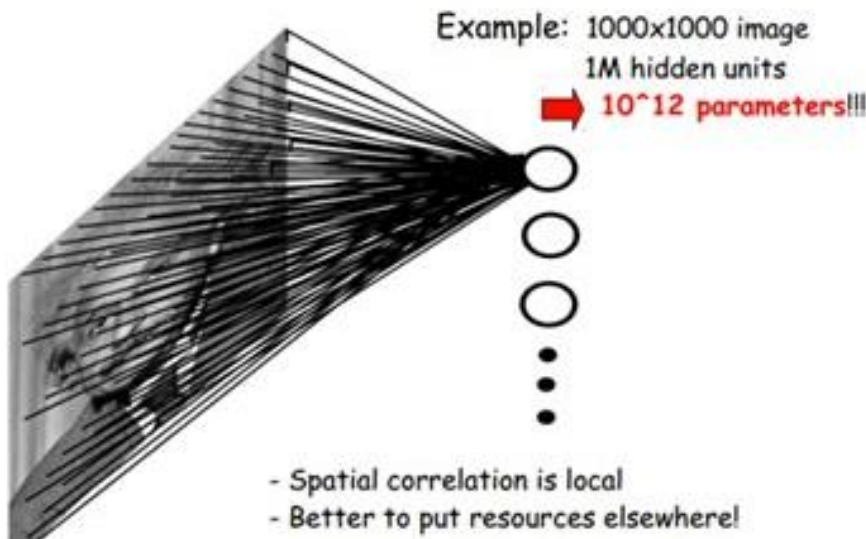


A closer look to CNN

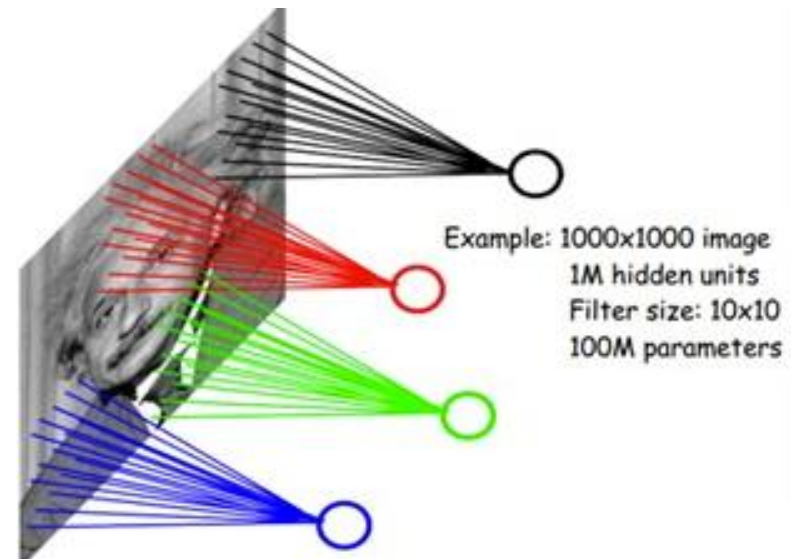
A thick, hand-drawn style orange line underlining the title.

Fully connectivity vs. Convolution

- The full connectivity in MLPs makes them **not scale well to higher resolution images**.
- MLPs **neglect the spatial structure of data**.
 - Pixels either far apart or close together are treated in the same way.



Fully connected neural network



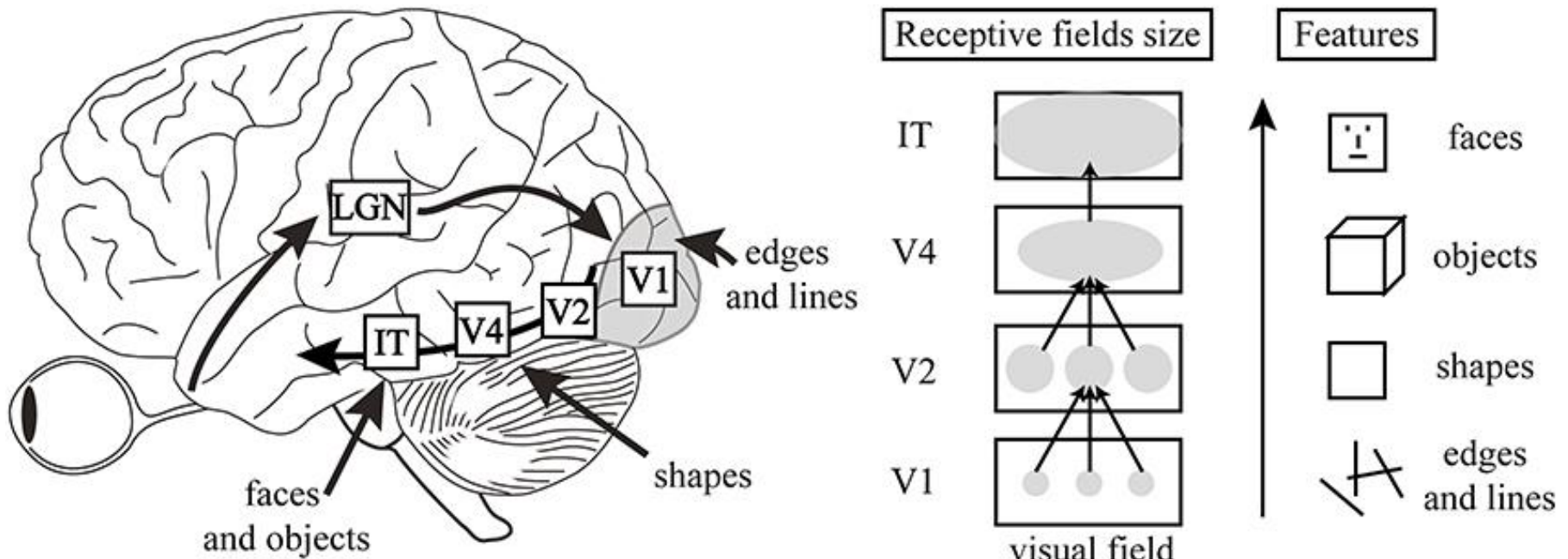
Locally connected neural network

Motivations for convolution

- Local connectivity
 - A neuron depends only on a few local input neurons
 - Translation invariance
- Comparison to Fully connected
 - Parameter sharing → reduce overfitting
 - Make use of spatial structure: **strong prior** for vision!
- Animal Vision Analogy
 - Hubel & Wiesel, Receptive fields of single neurons in the cat's striate cortex (1959)

Biological connection of CNNs

- The idea of **specialized components** inside of a system **having specific tasks** is the basis behind CNNs.
 - E.g., some neurons fired when exposed to vertical edges and some when shown horizontal or diagonal edges.

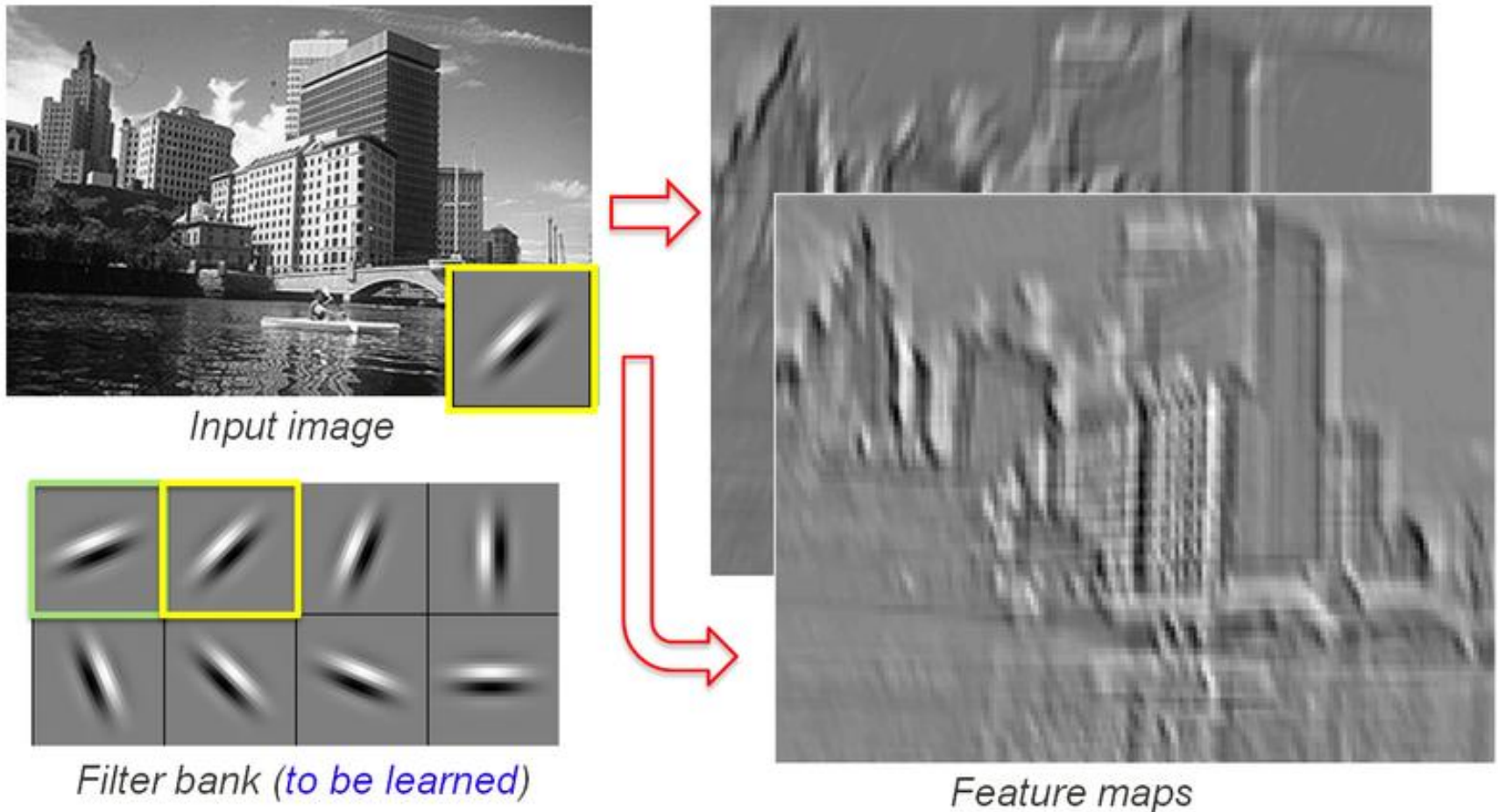


Hierarchical features in CNN

- CNNs learn the filters **automatically** in a **hierarchical** manner.
- The output of the 1st conv layer becomes the input of the 2nd conv layer, and so on.
 - E.g., a square is built from several straight edges, a semicircle is a combination of a curve and straight edge, etc.

The effects of multiple filters

- Each separate filters can discover a different pattern



The effects of multiple filters



Input

Input Volume (+pad 1) (7x7x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	1	1	2	0	2	0
0	1	0	1	1	0	0

0	0	0	1	2	1	0
0	1	1	1	2	2	0
0	0	2	0	0	2	0
0	0	0	0	0	0	0

$x[:, :, 1]$

0	0	0	0	0	0	0
0	1	2	1	1	0	0
0	0	0	2	0	0	0

0	1	0	0	2	2	0
0	1	2	1	0	2	0
0	0	0	1	2	2	0
0	0	0	0	0	0	0

$x[:, :, 2]$

0	0	0	0	0	0	0
0	1	0	2	2	0	0
0	2	1	0	1	0	0

0	0	0	1	1	0	0
0	0	1	2	1	0	0
0	2	1	0	0	2	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

$w0[:, :, 0]$

0	1	0
0	1	-1
-1	0	-1

$w0[:, :, 1]$

1	-1	-1
1	-1	1
-1	0	-1

$w0[:, :, 2]$

-1	1	1
0	1	1
1	-1	-1

Bias $b0$ (1x1x1)

$b0[:, :, 0]$

1

Filter W1 (3x3x3)

$w1[:, :, 0]$

-1	0	1
1	0	1
0	-1	1

$w1[:, :, 1]$

1	-1	1
1	1	0
1	0	0

$w1[:, :, 2]$

1	1	1
1	0	1
1	0	0

Bias $b1$ (1x1x1)

$b1[:, :, 0]$

0

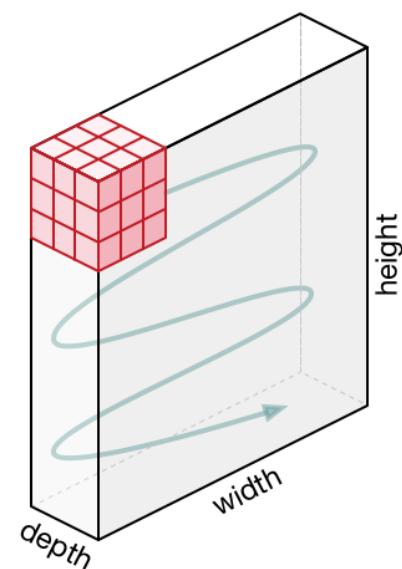
Output Volume (3x3x2)

$o[:, :, 0]$

0	8	4
0	-4	0
1	6	4

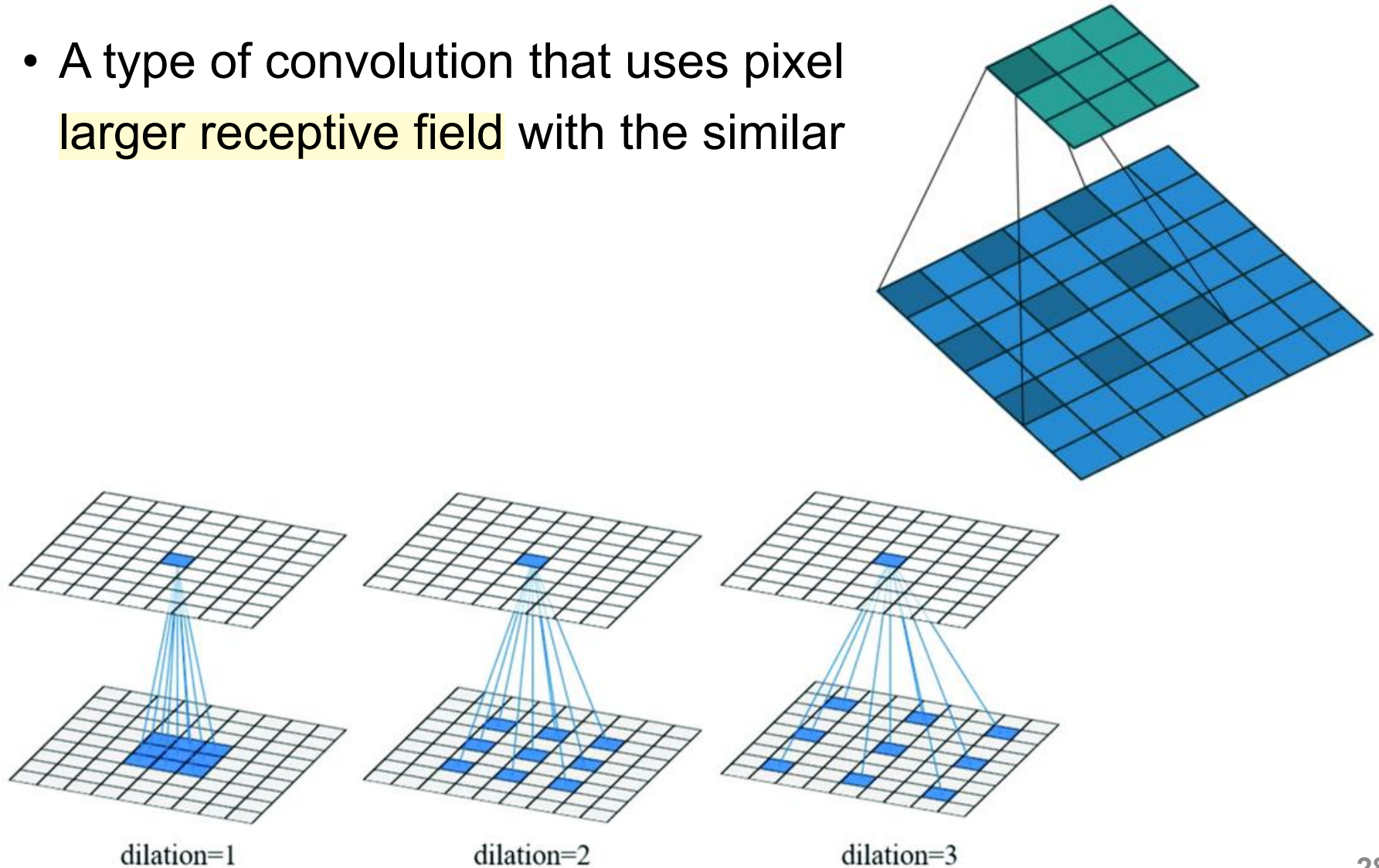
$o[:, :, 1]$

1	7	4
4	8	6
6	10	1



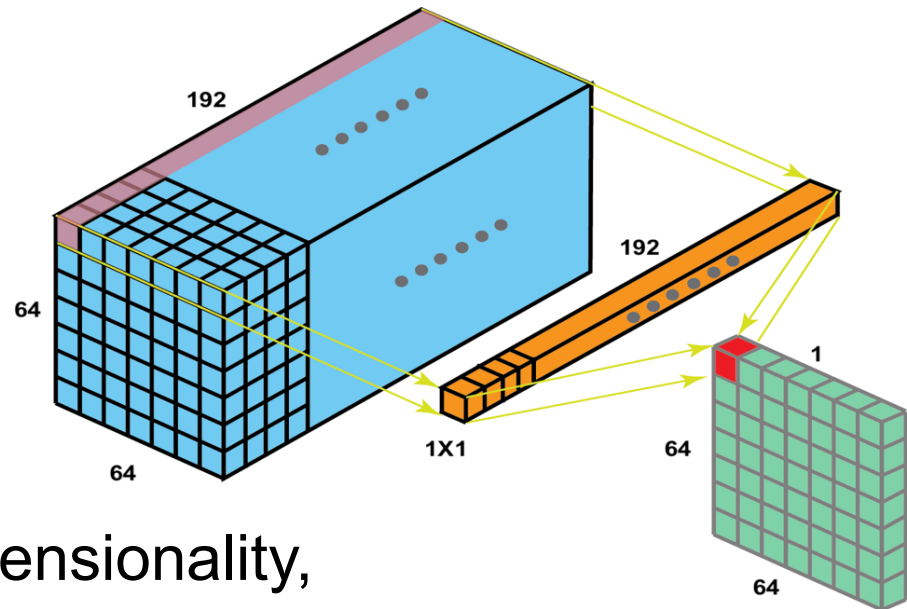
Dilated convolutional layer

- A type of convolution that uses pixel **larger receptive field** with the similar



1×1 convolutional filters

- **Goal:** change the dimensionality in the filter space.
- Consider a feature block of $W \times H \times D$, which is fed into a conv layer with D_1 1×1 filters, zero padding and stride 1.



- $D_1 > D$: increase dimensionality,
- or decrease otherwise.

Cross-entropy loss

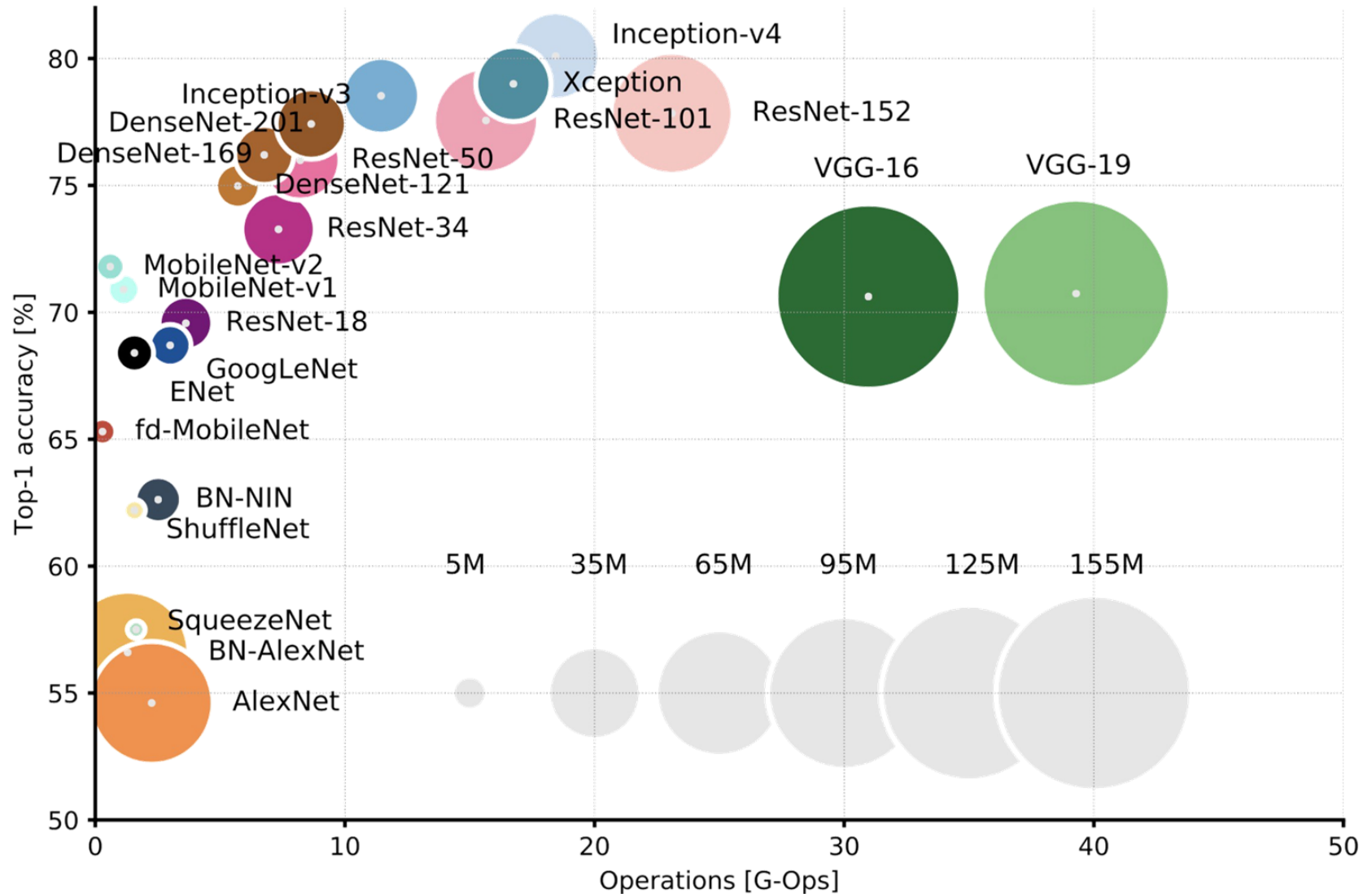
- **Cross-entropy loss** (or log loss) measures the performance of a classification model whose output is a probability value.

$$H(p, q) = H_q(p) = \sum_x p(x) \log \left(\frac{1}{q(x)} \right) = - \sum_x p(x) \log(q(x))$$

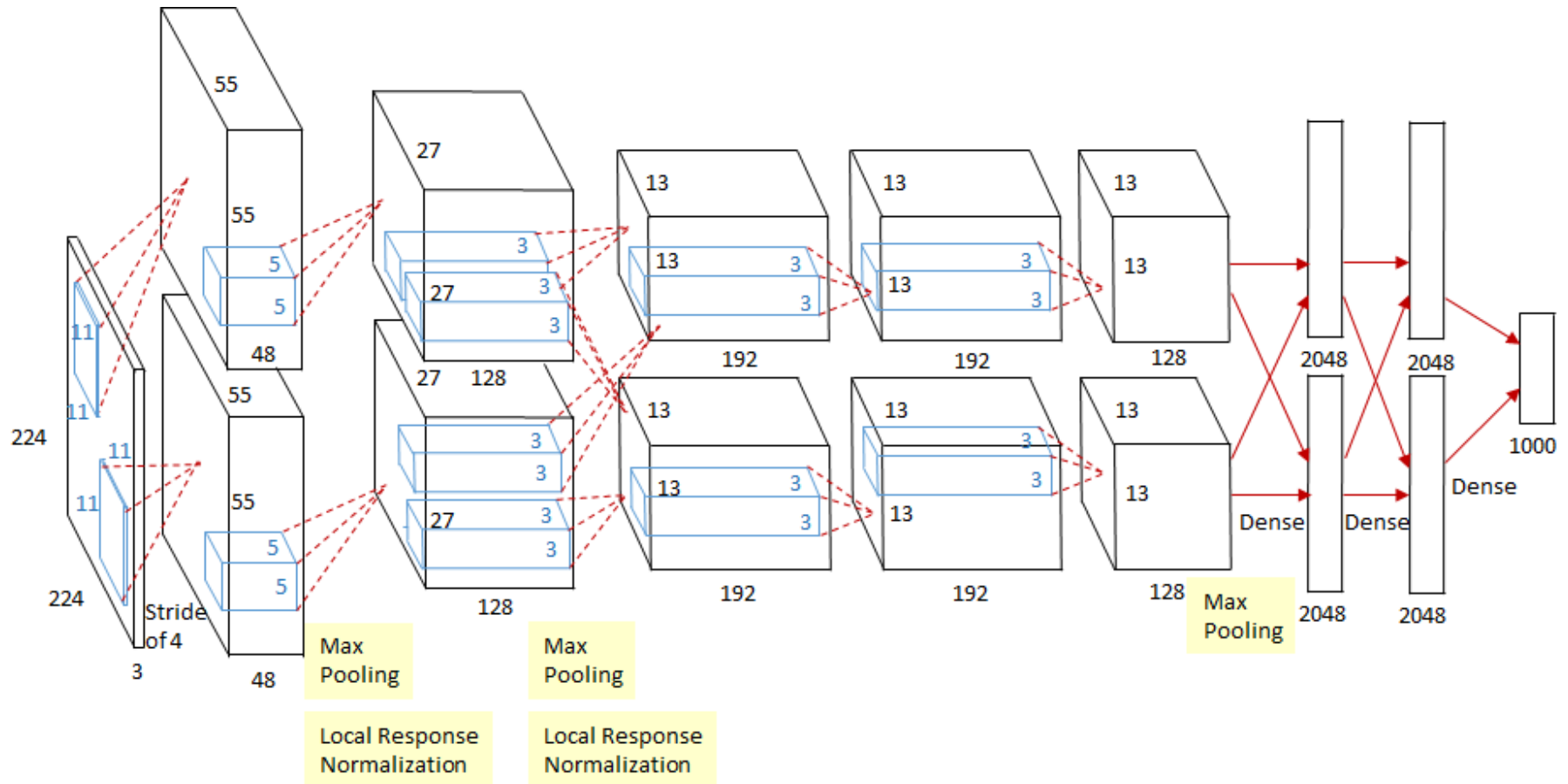
- $p(x)$ and $q(x)$ are the probabilities of x in distributions p and q .
- A detailed explanation can be found [here](#).
- In binary classification, the cross-entropy can be simply calculated as

$$\mathcal{L} = -y \log(p) + (1 - y) \log(1 - p)$$

A comparison of CNN architectures

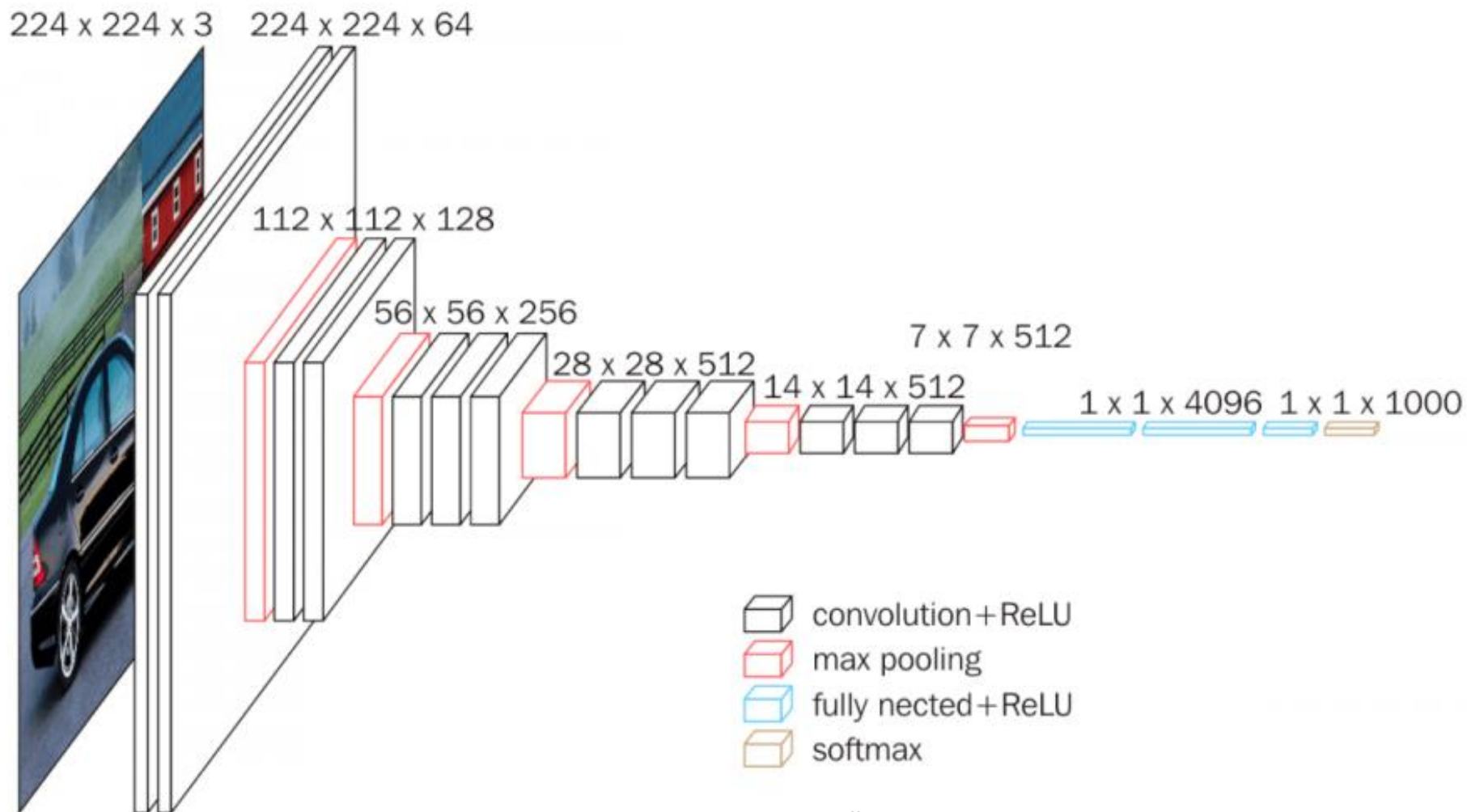


AlexNet architecture (2012)



Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems* 25 (2012).

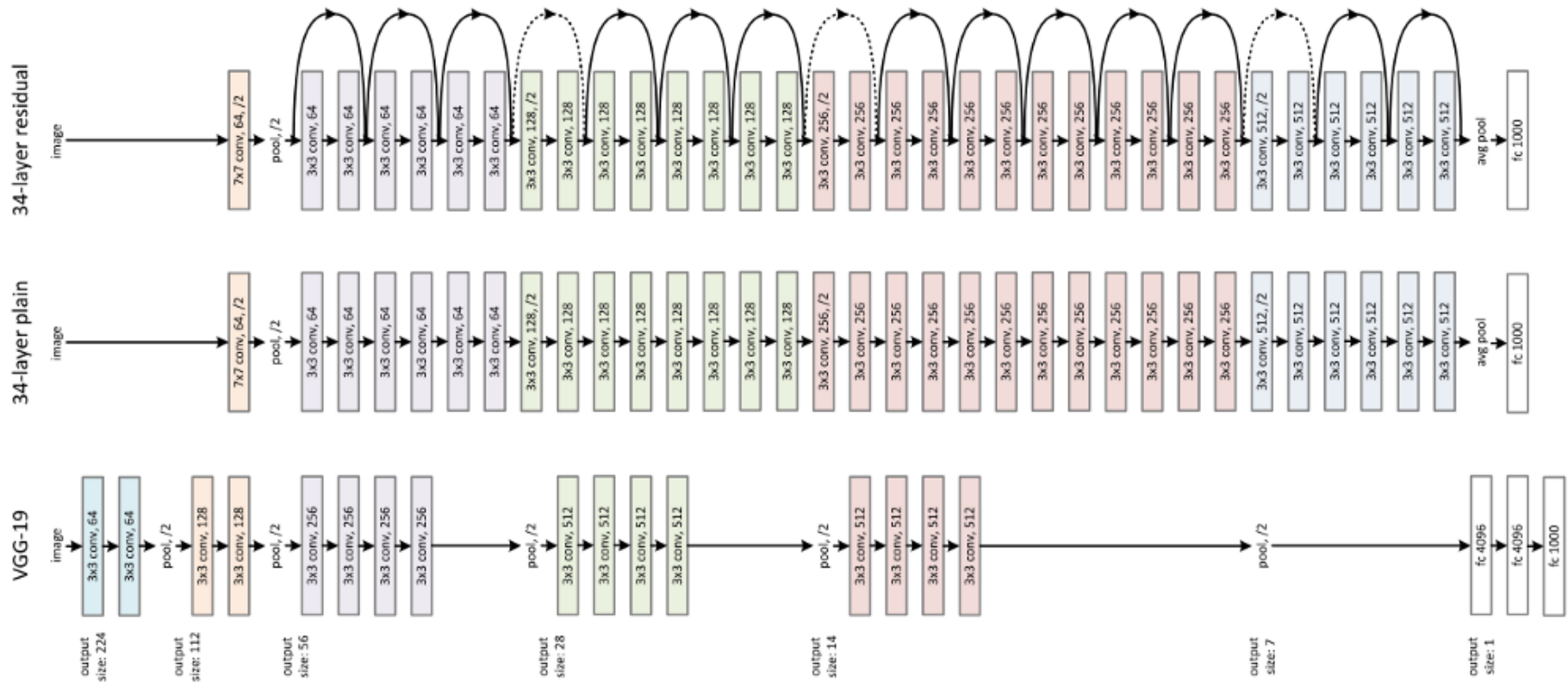
VGG16 architecture (2014)



Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." arXiv preprint arXiv:1409.1556 (2014).

ResNet architecture (2016)

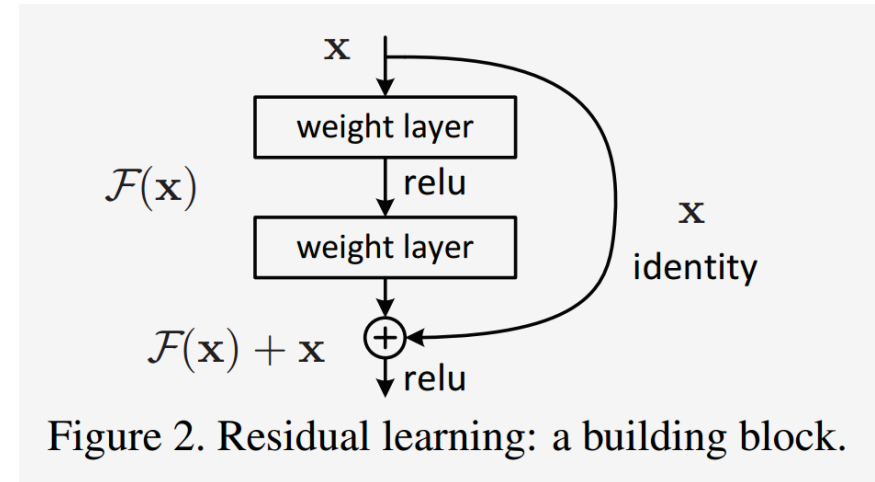
- Even deeper models: 34, 50, 101, 152 layers



He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep residual learning for image recognition." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770-778. 2016.

ResNet architecture (2016)

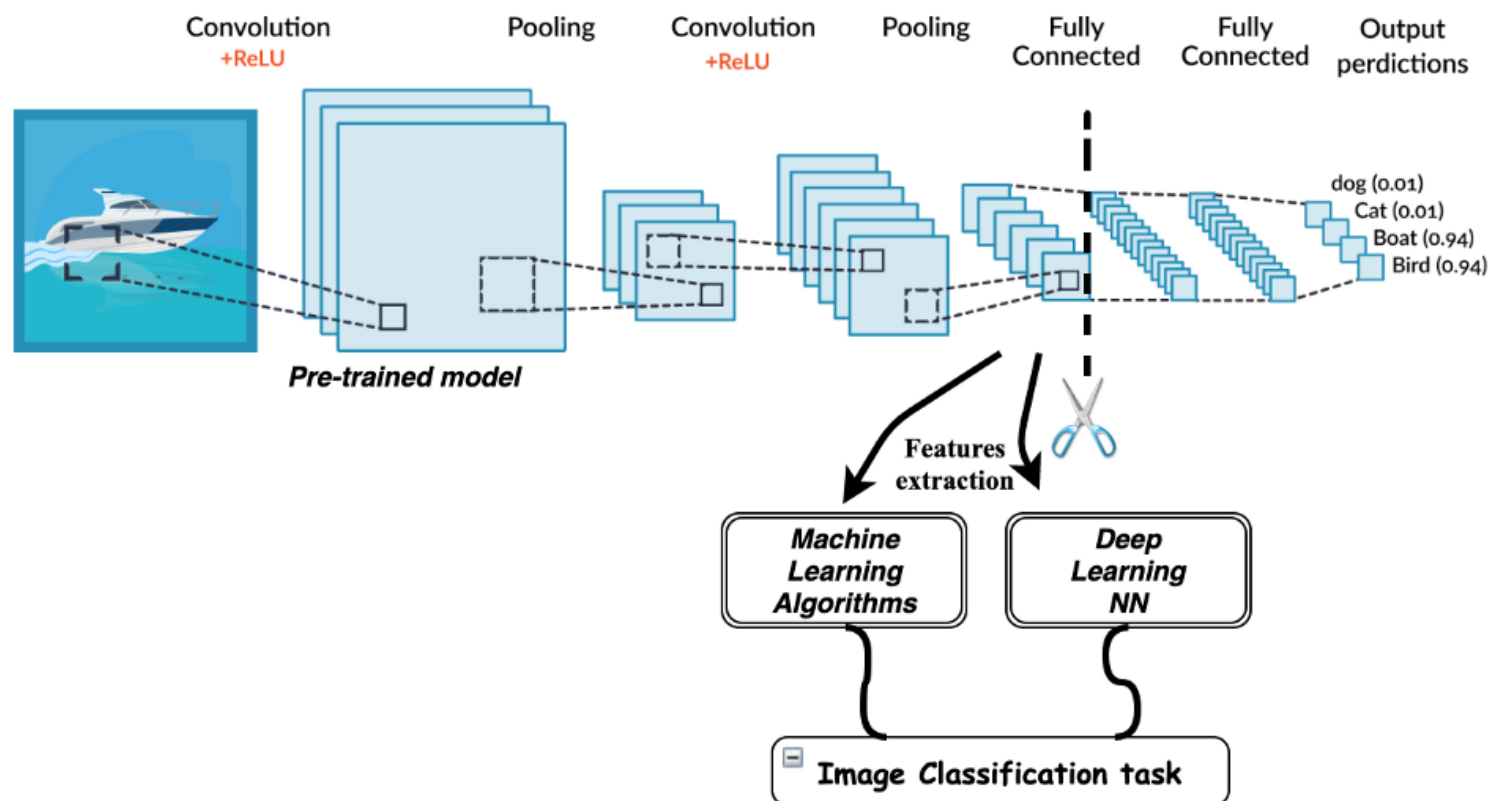
- A block learns the residual with responds to identity.
- Good optimization properties



- ResNet50 Compared to VGG
 - Superior accuracy in all vision tasks **5.25%** top-5 error vs 7.1%
 - Less parameters: **25M** vs 138M
 - Computational complexity: **3.8B Flops** vs 15.3B Flops
 - Fully Convolutional until the last layer

Pretrained CNN models: Transfer learning

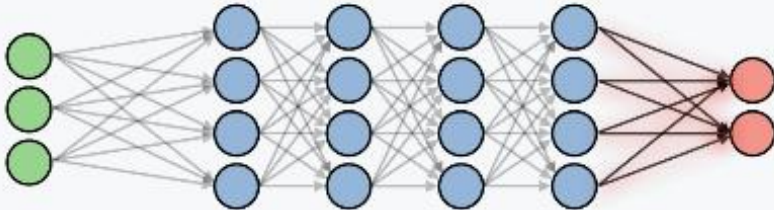
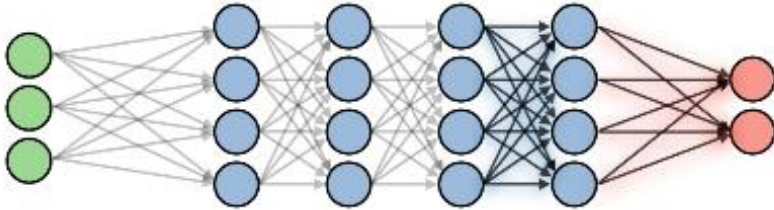
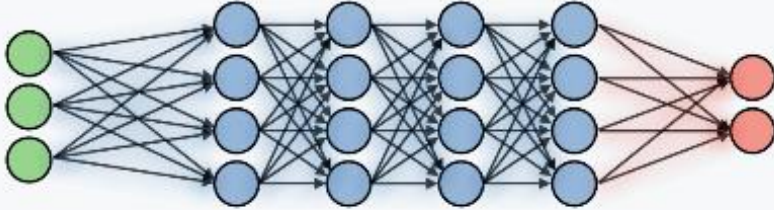
- Build models from scratch on large benchmarks (e.g., ImageNet) takes days or weeks while many of them are publicly available!
- Use pre-trained weights and remove the last layers to get features



Pretrained CNN models: Fine-tuning

- Retrain (some) parameters of the network (if enough data)
- Truncate the last layer(s) of the pre-trained network
- Freeze the remaining layers weights
- Add a (linear) classifier on top and train it for a few epochs
- Then fine-tune the whole network or the few deepest layers
- Use a smaller learning rate when fine tuning

Pretrained CNN models: Fine-tuning

Training size	Illustration	Explanation
Small		Freezes all layers, trains weights on softmax
Medium		Freezes most layers, trains weights on last layers and softmax
Large		Trains weights on layers and softmax by initializing weights on pre-trained ones

Acknowledgements

- Some parts of the slide are adapted from
 - Convolutional Neural Networks cheatsheet [Link](#)
 - A Beginner's Guide To Understanding Convolutional Neural Networks [Link](#)
 - CS231n: Convolutional Neural Networks for Visual Recognition [Link](#)
 - Convolutional Neural Network – Deep Learning Lectures [Link](#)
 - An Interactive Node-Link Visualization of Convolutional Neural Networks [Link](#)



...the end.

